

PROJECT REPORT

EXPERIMENT - 9

Project Statement

Design and simulate a Smart Flood Level Monitoring System using Ultrasonic Sensor. The system continuously monitors water levels and provides alerts during critical flood conditions for disaster management applications

Submitted by

Abinaya S – 24MER001

Dhiyanasree KK – 24MER008

Puthiyaraj M – 24MER045

Rithick S – 24MER051

Smart Flood Level Monitoring System Using Ultrasonic Sensor

Project Overview

The Smart Flood Level Monitoring System Using Ultrasonic Sensor is an IoT-based safety and disaster-management system designed to continuously monitor the water level in rivers, tanks, drainage channels, reservoirs, and other flood-prone areas.

The system uses an HC-SR04 ultrasonic sensor to measure the distance between the sensor and the water surface. An ESP8266 Node MCU processes the measured distance and determines the current flood-level condition. Two LED indicators are used to provide visual warnings for different water-level conditions.

When the water level reaches a critical condition, the system provides an immediate visual warning through the LED indicator. This helps in early identification of dangerous water levels and can support flood monitoring and disaster-management applications.

Main Objectives:

The main objectives of the Smart Flood Level Monitoring System are:

- To continuously monitor the water level using an ultrasonic sensor.
- To measure the distance between the sensor and the water surface.
- To process the sensor readings using an ESP8266 Node MCU.
- To identify normal and critical flood conditions.
- To provide visual alerts using LED indicators.
- To develop a low-cost and simple flood-monitoring prototype.
- To demonstrate the application of IoT technology in disaster management.
- To reduce the need for continuous manual water-level monitoring.

Problem Statement

Floods can cause significant damage to people, buildings, roads, agricultural land, and infrastructure. Conventional water-level monitoring methods may require manual observation and may not provide timely warnings during rapidly changing water conditions.

Therefore, there is a need for an automated system that can continuously monitor water levels and provide an early warning when the water reaches a critical level.

The proposed Smart Flood Level Monitoring System uses an HC-SR04 ultrasonic sensor and ESP8266 Node MCU to detect water-level changes and provide LED-based alerts during unsafe conditions.

Problems Addressed:

The proposed system addresses the following problems:

- Manual monitoring of water levels.
- Delayed identification of rising water levels.
- Lack of continuous monitoring.
- Difficulty in monitoring flood-prone locations.
- Requirement for a low-cost monitoring solution.
- Need for quick visual indication of critical water levels.
- Increased risk caused by sudden water-level changes.

Proposed Solution

The proposed system consists of an ESP8266 Node MCU, HC-SR04 ultrasonic sensor, and two LED indicators.

The ultrasonic sensor is positioned above the water surface. It sends ultrasonic waves toward the water and receives the reflected signal. The ESP8266 calculates the distance based on the time taken by the ultrasonic wave to return.

As the water level rises, the measured distance decreases. The Node MCU compares the measured distance with predefined threshold values.

Proposed Operation:

The system operates in the following sequence:

- The HC-SR04 ultrasonic sensor is mounted above the monitored water body.
- The sensor sends an ultrasonic pulse toward the water surface.
- The pulse is reflected back from the water surface.
- The HC-SR04 calculates the time taken for the echo to return.
- The ESP8266 receives the echo information.
- The Node MCU calculates the distance to the water surface.
- The calculated distance is compared with predefined threshold values.
- If the water level is within the safe range, the normal-level LED is activated.
- If the water rises above the critical threshold, the warning LED is activated.
- The process repeats continuously to monitor changing water levels.

System Architecture

Input Section:

HC-SR04 Ultrasonic Sensor: The ultrasonic sensor measures the distance between the sensor and the water surface.

Processing Section:

ESP8266 Node MCU: The Node MCU receives the ultrasonic sensor output, calculates the water-surface distance, and compares it with predefined threshold values.

Indication Section:

LED Indicators: Two LEDs indicate the water-level condition:

- Normal/Safe condition
- Critical flood condition

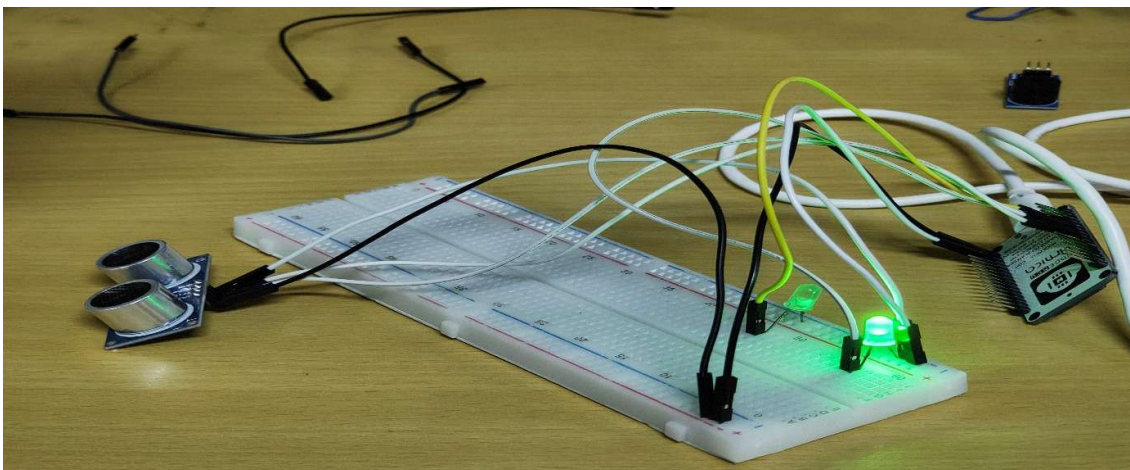
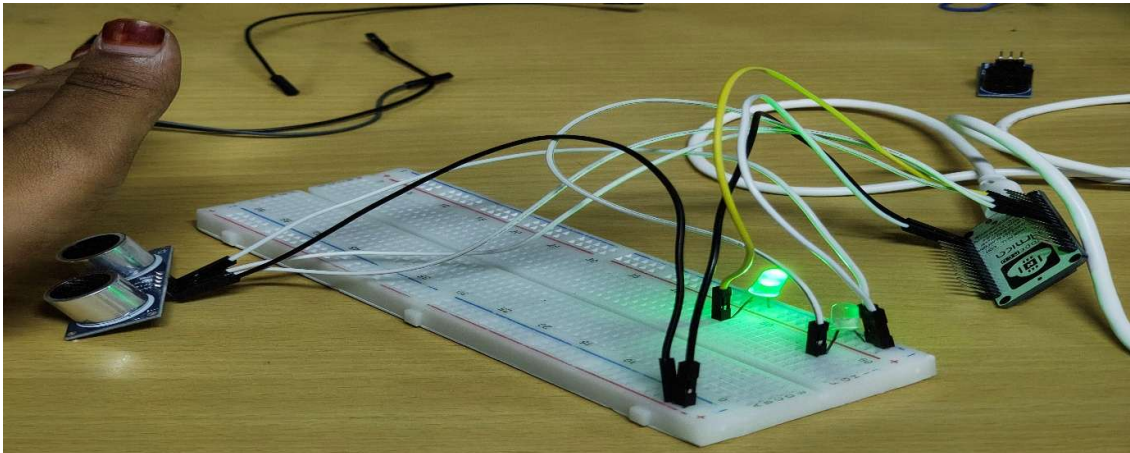
Power Section:

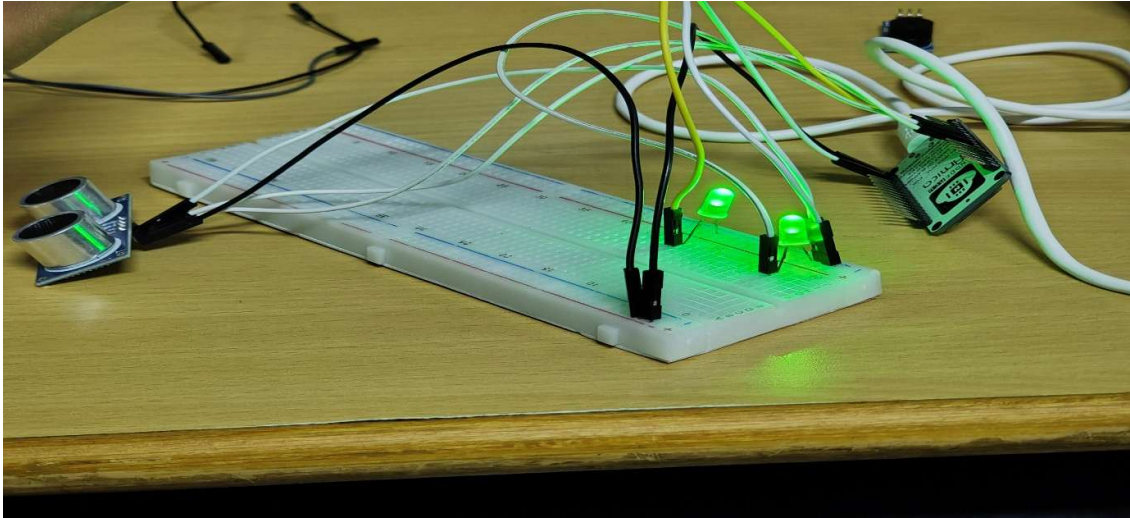
The Node MCU and connected components are powered using a suitable USB power supply.

Overall Block Flow:

Water Surface → HC-SR04 Ultrasonic Sensor → ESP8266 Node MCU → Water-Level Decision → LED Indicators

Image:

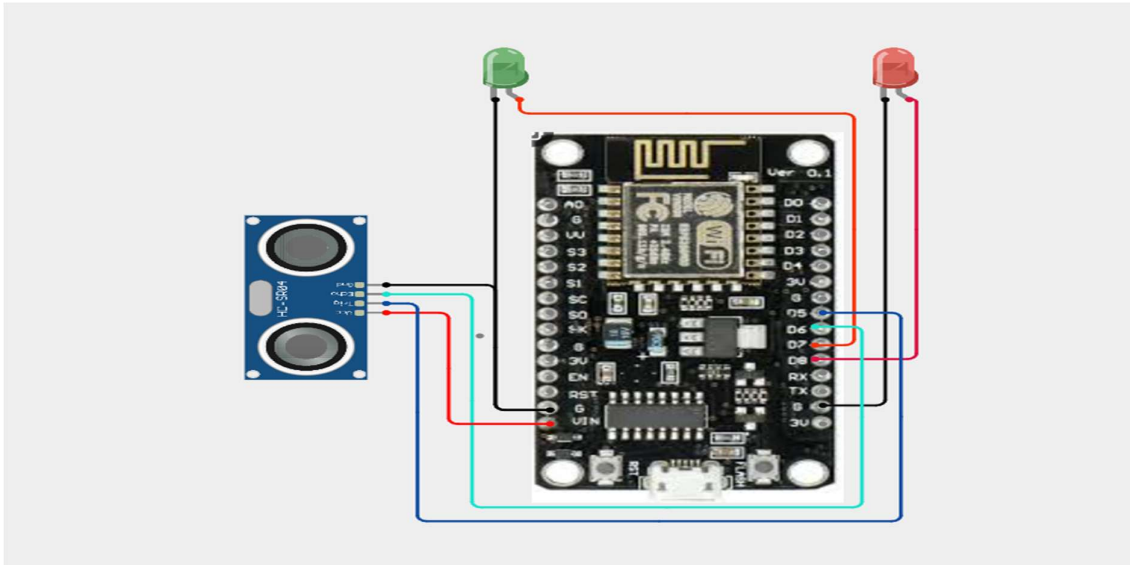




Circuit Table

S. No.	Component	Pin	ESP8266 Node MCU Connection
1	HC-SR04 Ultrasonic Sensor	VCC	VIN / 5V
2	HC-SR04 Ultrasonic Sensor	GND	GND
3	HC-SR04 Ultrasonic Sensor	TRIG	D5 (GPIO14)
4	HC-SR04 Ultrasonic Sensor	ECHO	D6 (GPIO12) through voltage divider
5	Normal-Level LED	Anode (+)	D1 (GPIO5) through 220 Ω resistor
6	Normal-Level LED	Cathode (-)	GND
7	Critical-Level LED	Anode (+)	D2 (GPIO4) through 220 Ω resistor
8	Critical-Level LED	Cathode (-)	GND
9	Breadboard	Power Rail (+)	VIN / 5V
10	Breadboard	Ground Rail (-)	GND

Circuit Design



Important Pin Mapping:

A suitable example pin assignment is:

- HC-SR04 TRIG → D5 (GPIO14)
- HC-SR04 ECHO → D6 (GPIO12) through a voltage divider
- Normal LED → D1 (GPIO5) through a current-limiting resistor
- Critical LED → D2 (GPIO4) through a current-limiting resistor
- HC-SR04 VCC → 5V
- All GND terminals → Common GND

Algorithm

Step 1:Start the system.

Step 2:Initialize the ESP8266 Node MCU, ultrasonic sensor, and LED pins.

Step 3:Trigger the HC-SR04 sensor.

Step 4:Measure the echo return time.

Step 5:Calculate the distance between the sensor and water surface.

Step 6: Compare the measured distance with the predefined threshold.

Step 7: If the water is at a safe level, turn ON the normal-level LED.

Step 8: If the water reaches the critical level, turn OFF the normal LED and turn ON the critical LED.

Step 9: Display/process the current status as required.

Step 10: Repeat the measurement continuously.

Coding

```
#include <ESP8266WiFi.h>

#include <ThingSpeak.h>

// --- Wi-Fi Settings ---

const char* WIFI_SSID = "Rithick";
const char* WIFI_PASS = "12345678";

// --- ThingSpeak Settings ---

WiFiClient client;

unsigned long myChannelNumber = 3460914;

const char * myWriteAPIKey = "T0JP2HWN8I3YDEYF";

// --- ThingSpeak Fields ---

// Field 1: Distance (cm)

// Field 2: Alert Level (0 = Safe, 1 = Warning, 2 = Critical)

// --- Pin Definitions ---

const int TRIG_PIN = 14; // D5 (GPIO 14) -> HC-SR04 Trigger
```



```
const int ECHO_PIN = 12; // D6 (GPIO 12) -> HC-SR04 Echo (Via Voltage Divider)
```

```
const int LED_GREEN = 13; // D7 (GPIO 13) -> Green LED (+ 220 Ohm resistor)
```

```
const int LED_RED = 15; // D8 (GPIO 15) -> Red LED (+ 220 Ohm resistor)
```

```
// --- Threshold Configurations (cm) ---
```

```
const float DIST_CRITICAL = 10.0;
```

```
const float DIST_WARNING = 30.0;
```

```
// --- Timing Variables ---
```

```
unsigned long lastThingSpeakUpdate = 0;
```

```
const unsigned long THINGSPEAK_INTERVAL = 15000; // Free accounts  
require >= 15 seconds
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    delay(10);
```

```
    pinMode(TRIG_PIN, OUTPUT);
```

```
    pinMode(ECHO_PIN, INPUT);
```

```
    pinMode(LED_GREEN, OUTPUT);
```

```
    pinMode(LED_RED, OUTPUT);
```

```
    digitalWrite(TRIG_PIN, LOW);
```

```
    digitalWrite(LED_GREEN, LOW);
```

```
    digitalWrite(LED_RED, LOW);
```

```
Serial.println("\n-----");  
Serial.println(" ESP8266 Ultrasonic Sensor & ThingSpeak Log");  
Serial.println("-----");
```

```
// Connect to Wi-Fi
```

```
connectWiFi();
```

```
// Initialize ThingSpeak API client
```

```
ThingSpeak.begin(client);
```

```
}
```

```
void loop() {
```

```
  // Ensure Wi-Fi stays connected
```

```
  if (WiFi.status() != WL_CONNECTED) {
```

```
    connectWiFi();
```

```
  }
```

```
// 1. Read Sensor
```

```
float distanceCm = readDistance();
```

```
// 2. Local Alerts & State Tracking
```

```
int alertLevel = 0; // Default: Safe
```

```
handleAlerts(distanceCm, alertLevel);
```

```
// 3. Serial Monitor Output
```

```
Serial.print("Distance: ");  
if (distanceCm < 0) {  
    Serial.print("Out of Range / Error");  
} else {  
    Serial.print(distanceCm, 1);  
    Serial.print(" cm");  
}
```

```
Serial.print(" | Status: ");  
if (alertLevel == 2) {  
    Serial.println("[CRITICAL]");  
} else if (alertLevel == 1) {  
    Serial.println("[WARNING]");  
} else {  
    Serial.println("[SAFE]");  
}
```

```
// 4. Periodic Logging to ThingSpeak
```

```
unsigned long currentMillis = millis();  
if (currentMillis - lastThingSpeakUpdate >= THINGSPEAK_INTERVAL) {  
    lastThingSpeakUpdate = currentMillis;  
    sendToThingSpeak(distanceCm, alertLevel);  
}  
  
delay(150);  
}
```

```
float readDistance() {  
    digitalWrite(TRIG_PIN, LOW);  
    delayMicroseconds(2);  
    digitalWrite(TRIG_PIN, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(TRIG_PIN, LOW);  
  
    long duration = pulseIn(ECHO_PIN, HIGH, 30000);  
    if (duration == 0) return -1.0;  
  
    return (duration * 0.0343) / 2.0;  
}
```

```
void handleAlerts(float distance, int &alertLevel) {  
    // CRITICAL ZONE (< 10 cm) -> Red LED ON, Green LED OFF  
    if (distance > 0 && distance < DIST_CRITICAL) {  
        digitalWrite(LED_RED, HIGH);  
        digitalWrite(LED_GREEN, LOW);  
        alertLevel = 2; // Critical  
    }  
    // WARNING ZONE (10 cm - 30 cm) -> Both LEDs ON  
    else if (distance >= DIST_CRITICAL && distance < DIST_WARNING) {  
        digitalWrite(LED_RED, HIGH);  
        digitalWrite(LED_GREEN, HIGH);  
        alertLevel = 1; // Warning  
    }  
}
```

```
}  
  
// SAFE ZONE (> 30 cm or Out of Range) -> Green LED ON, Red LED OFF  
  
else {  
    digitalWrite(LED_RED, LOW);  
    digitalWrite(LED_GREEN, HIGH);  
    alertLevel = 0; // Safe  
}  
}
```

```
void connectWiFi() {  
    Serial.print("Connecting to Wi-Fi: ");  
    Serial.println(WIFI_SSID);  
    WiFi.mode(WIFI_STA);  
    WiFi.begin(WIFI_SSID, WIFI_PASS);
```

```
    int attempts = 0;  
    while (WiFi.status() != WL_CONNECTED && attempts < 20) {  
        delay(500);  
        Serial.print(".");  
        attempts++;  
    }
```

```
    if (WiFi.status() == WL_CONNECTED) {  
        Serial.println("\nWiFi Connected!");  
        Serial.print("IP Address: ");  
        Serial.println(WiFi.localIP());
```

```
    } else {  
        Serial.println("\nWiFi Connection Failed. Proceeding locally...");  
    }  
}
```

```
void sendToThingSpeak(float distance, int alertLevel) {
```

```
    if (WiFi.status() != WL_CONNECTED) return;
```

```
    Serial.println("---> Uploading to ThingSpeak...");
```

```
    // Set values for ThingSpeak fields
```

```
    ThingSpeak.setField(1, distance);
```

```
    ThingSpeak.setField(2, alertLevel);
```

```
    int    responseCode    =    ThingSpeak.writeFields(myChannelNumber,  
myWriteAPIKey);
```

```
    if (responseCode == 200) {
```

```
        Serial.println("---> ThingSpeak upload successful (HTTP 200)");
```

```
    } else {
```

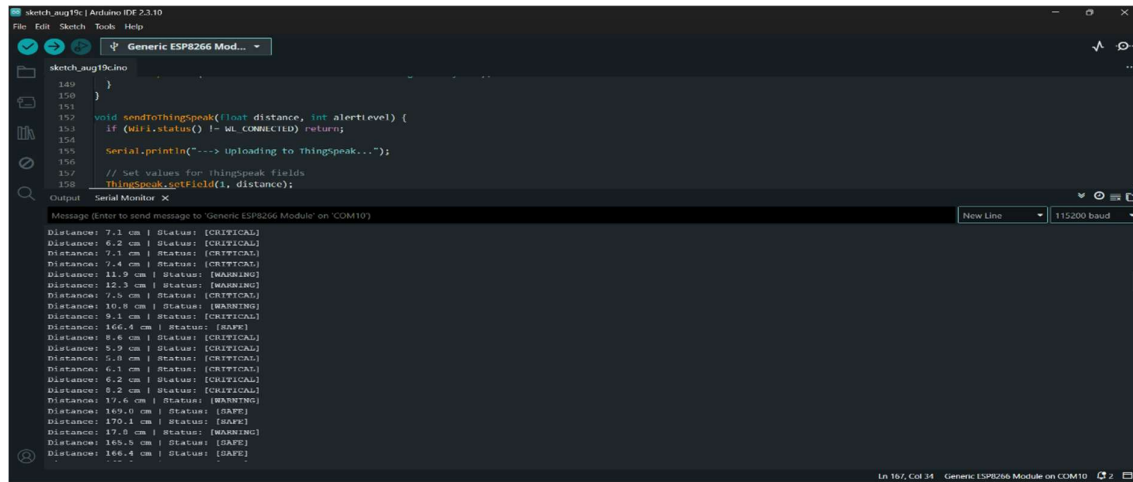
```
        Serial.print("---> Error uploading to ThingSpeak. Code: ");
```

```
        Serial.println(responseCode);
```

```
    }
```

```
}
```

Coding Output:



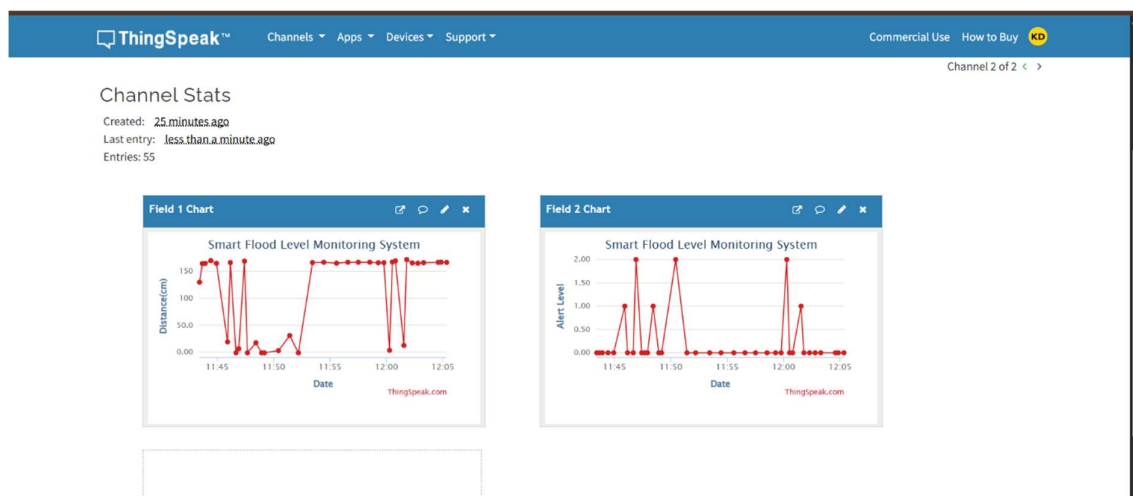
```
sketch_aug19c.ino
149 }
150
151
152 void sendToThingspeak(float distance, int alertlevel) {
153   if (mqtt.status() != ML_CONNECTED) return;
154   Serial.println("----> uploading to Thingspeak...");
155   // Set values for thingspeak fields
156   Thingspeak.setField(1, distance);
157 }
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Serial Monitor X

Message (Enter to send message to 'Generic ESP8266 Module' on 'COM10')

New Line 115200 baud

Distance: 7.1 cm | Status: [CRITICAL]
Distance: 6.2 cm | Status: [CRITICAL]
Distance: 7.1 cm | Status: [CRITICAL]
Distance: 7.4 cm | Status: [CRITICAL]
Distance: 11.9 cm | Status: [WARNING]
Distance: 12.3 cm | Status: [WARNING]
Distance: 7.5 cm | Status: [CRITICAL]
Distance: 10.8 cm | Status: [WARNING]
Distance: 9.1 cm | Status: [CRITICAL]
Distance: 166.4 cm | Status: [SAFE]
Distance: 8.4 cm | Status: [CRITICAL]
Distance: 5.9 cm | Status: [CRITICAL]
Distance: 5.8 cm | Status: [CRITICAL]
Distance: 6.1 cm | Status: [CRITICAL]
Distance: 6.2 cm | Status: [CRITICAL]
Distance: 8.2 cm | Status: [CRITICAL]
Distance: 17.4 cm | Status: [WARNING]
Distance: 169.0 cm | Status: [SAFE]
Distance: 170.1 cm | Status: [SAFE]
Distance: 17.5 cm | Status: [WARNING]
Distance: 165.5 cm | Status: [SAFE]
Distance: 166.4 cm | Status: [SAFE]



System Working

The HC-SR04 ultrasonic sensor is installed above the water surface at a fixed height. The sensor periodically emits ultrasonic waves toward the water.

When the waves hit the water surface, they are reflected toward the sensor. The time required for the ultrasonic signal to travel to the water surface and return is measured.

The ESP8266 uses this time to calculate the distance.

The relationship can be represented as:

$$\text{Distance} = (\text{Speed of Sound} \times \text{Echo Time}) / 2$$

The division by 2 is required because the ultrasonic wave travels from the sensor to the water surface and then returns to the sensor.

As the water level rises, the distance measured by the sensor decreases. The Node MCU compares this value with the selected threshold.

Bill of Materials

S.No.	Component	Specification	Quantity
1	ESP8266 Node MCU	Wi-Fi-enabled development board	1
2	HC-SR04 Ultrasonic Sensor	2–400 cm range	1
3	LED Indicator – Green	5 mm LED	1
4	LED Indicator – Red	5 mm LED	1
5	Resistor	220 Ω	2
6	Resistor	Suitable values for Echo voltage divider	2
7	Breadboard	Standard solderless breadboard	1
8	Jumper Wires	Male–male / male–female	1 set
9	USB Cable	Micro-USB, compatible with NodeMCU	1
10	5 V Power Source	USB power supply	1

Prototype Implementation

The prototype is constructed on a breadboard using the ESP8266 Node MCU as the central controller.

The HC-SR04 ultrasonic sensor is positioned above a container containing water. The sensor is mounted at a fixed height so that changes in water level can be detected accurately.

Two LEDs are connected to the Node MCU to indicate the operating condition. One LED represents the normal water-level condition, while the second LED represents the critical flood condition.

The HC-SR04 Echo signal is connected to the Node MCU through a suitable voltage-divider circuit because the ESP8266 operates using 3.3 V GPIO logic.

Hardware Setup:

The hardware setup consists of the following steps:

- Place the ESP8266 Node MCU on the breadboard.
- Mount the HC-SR04 sensor above the water container.
- Connect the HC-SR04 VCC to the appropriate supply.
- Connect the HC-SR04 GND to common ground.
- Connect the TRIG pin to the selected Node MCU GPIO.
- Connect the ECHO pin through a voltage divider to the selected Node MCU GPIO.
- Connect the normal LED to the assigned GPIO through a resistor.
- Connect the critical LED to another GPIO through a resistor.
- Connect all grounds together.
- Power the Node MCU using a USB cable.
- Upload the monitoring program to the ESP8266.
- Test the system by gradually changing the water level.

Prototype Working

During prototype operation, the ultrasonic sensor continuously measures the distance from its position to the water surface.

When the water level is low, the measured distance is relatively large and the system remains in the safe condition.

As water is added to the container, the water surface moves closer to the ultrasonic sensor. Therefore, the measured distance decreases.

When the measured distance reaches the predefined critical threshold, the ESP8266 changes the system status and activates the critical LED.

This demonstrates how the prototype can identify a rising water level and provide an early visual warning.

Expected Prototype Output

Water-Level Condition	Ultrasonic Sensor Reading	Expected LED Output	System Status
Safe Level	Distance above critical threshold	Green LED ON	SAFE
Rising Water Level	Distance gradually decreases	Green LED remains ON	MONITOR
Critical Level	Distance reaches critical threshold	Red LED ON	FLOOD ALERT
Water Level Falls	Distance increases above threshold	Green LED ON	SAFE

Prototype Demonstration

The prototype can be demonstrated using a transparent water container.

Demonstration Procedure:

- Place the ultrasonic sensor above the empty or partially filled container.
- Power ON the ESP8266.
- Observe the normal-level LED.
- Gradually add water to the container.
- Observe the reduction in measured distance.
- Continue increasing the water level until the critical threshold is reached.
- Observe the critical LED activation.
- Remove some water and verify that the system returns to the normal condition.

Results & Performance Analysis

The developed prototype successfully demonstrates automatic water-level monitoring using an ultrasonic sensing technique.

The HC-SR04 provides distance measurements corresponding to changes in the water surface position. The ESP8266 processes these measurements and determines whether the water level is within the safe range or has reached the critical threshold.

Expected Results:

The expected results are:

- Accurate detection of changing water levels within the prototype setup.
- Continuous monitoring without manual measurement.
- Automatic identification of critical water-level conditions.
- Immediate LED-based visual warning.

- Reliable operation of the ESP8266 controller.
- Low-cost and compact prototype implementation.
- Demonstration of IoT-based technology for disaster-management applications.

Performance Analysis:

Parameter	Expected Performance	Observation
Water-Level Detection	Detects changes in water level	Water-level changes can be detected continuously
Measurement Method	Non-contact measurement	Sensor measures the water surface without physical contact
Response Time	Fast response	LED status changes shortly after the threshold is reached
Critical-Level Detection	Automatic	Red LED activates when the critical threshold is reached
Safe-Level Indication	Automatic	Green LED indicates normal water conditions
Continuous Monitoring	Yes	Repeated ultrasonic measurements provide continuous monitoring
Controller Performance	Reliable	ESP8266 processes sensor readings effectively
System Cost	Low	Uses commonly available electronic components
Prototype Reliability	Good under controlled conditions	Performance depends on sensor positioning and water-surface stability
Power Requirement	Low	Suitable for continuous operation with a suitable USB supply